While RUM focuses on end users, there is another user-tracking category: internal users, including developers and operators. For example, you might want to have a record of the actions taken by a user or a service account for compliance or risk auditing reasons. An example of this kind of service is AWS CloudTrail (http://mng.bz/N2gN), supporting a long list of services, including Amazon API Gateway, AWS CloudFormation, Amazon CloudWatch, Amazon EKS, AWS Lambda, and AWS Systems Manager.

You can also set up alarms from the aforementioned internal-user audit logs. With that said, let's now move on to costs.

7.3.2 Costs

A totally different category of tracking access to resources is cost tracking. No matter if you want to educate internal users about costs and usage (showback) or offer services that require charging certain tenets (chargeback), you want to have an idea of what the actual costs are on as fine-grained a level as possible. We can distinguish between metering, which is counting the access (e.g., API endpoint XYZ was called 3,478 times in the last 15 minutes), and the actual charges that can depend on many things, from upfront commitments to special deals you may be able to negotiate based on the volume you are consuming.

Cloud providers typically offer services that allow you to (interactively) explore your usage (e.g., AWS Cost and Usage Reports [CUR], http://mng.bz/D46n). In addition to these native offerings, you may wish to opt into using solutions that provide for cost allocation and resource usage monitoring down to single units of execution, like OpenCost (https://www.opencost.io/) does for Kubernetes environments.

With this high-level discussion of user and usage tracking and reporting, we've reached the end of the cloud operations chapters. While most of the topics we've discussed are primarily important to understand and owned by folks in operator roles, with cloud-native systems, it's crucial that the people in every role involved, from developers to testers to release managers, have a solid grasp of the concepts. This can, in the case of a fully serverless environment, mean that, indeed, developers themselves are on call (http://mng.bz/lW76).

Summary

- To successfully operate cloud-native applications, you need to have a structured approach to detecting, handling, and learning from incidents.
- Health and performance monitoring allows you to have a system-external view and learn that something is not working as expected.
- If you're on call for a service and an incident happens, focus on stopping the bleeding; the analysis of what went wrong comes later, ideally in a blameless postmortem.
- The goal of alerting is for a human to automatically be notified of an event impacting the system.